

The Origamizer Algorithm

Algorithm Diagrams for Folding Any Polyhedron

Pseudocode formalization of

Erik D. Demaine & Tomohiro Tachi, *Origamizer*, SoCG 2017

This document renders each algorithmic stage of the *Origamizer* method as explicit pseudocode. The original paper describes the algorithm in prose (§3–§7); the section tags below cross-reference those sections. Symbols follow the paper: Q is the target oriented *polyhedral manifold* (disk topology, strictly convex facets, with a *clean* side and a *tuck* side); $\varepsilon > 0$ is the requested tolerance; T is the *tuck proxy* (a polyhedral *waffle* within ε of Q); and ε' with $0 < \varepsilon' < \varepsilon$ lower-bounds how far T may extend beyond Q . The goal is a convex piece of paper P and a *watertight, seamless, ε -extra* folding $f: P \rightarrow Q$ with $f(P) \subseteq T$.

Top-level driver

Algorithm 1: $\text{ORIGAMIZER}(Q, \varepsilon)$ — fold a convex paper polygon into Q

Input: Oriented polyhedral manifold Q (disk, strictly convex facets, clean/tuck sides); tolerance $\varepsilon > 0$.

Output: Convex polygon P and watertight, seamless, ε -extra folding $f: P \rightarrow Q$ with $f(P) \subseteq T$.

```
1  $(T, \varepsilon') \leftarrow \text{ComputeTuckProxy}(Q, \varepsilon)$ 
    $\triangleright$  §3: attach  $\varepsilon$ -thin walls; split reflex curvature
2  $\Sigma \leftarrow \text{SquashWafflePockets}(T, \varepsilon')$ 
    $\triangleright$  §4: flatten each waffle pocket; emit cells & perpendiculars
3  $(P', \mathcal{S}, \delta) \leftarrow \text{PlaceStreams}(T, \varepsilon', \Sigma)$ 
    $\triangleright$  §5: Tutte-embed waffle dual; draw rivers & brooks
4  $(P, \mathcal{V}) \leftarrow \text{PlaceSites}(T, \varepsilon', \Sigma, \mathcal{S})$ 
    $\triangleright$  §6: place point/segment/polygon sites; define  $P$ 
5  $f \leftarrow \text{VoronoiFolding}(T, P, \mathcal{V})$ 
    $\triangleright$  §7: fold  $P$  into the tuck proxy via an abstract waffle
6 return  $(P, f)$ 
```

§3 Tuck Proxy and Waffles

Computes the *tuck proxy* T : a polyhedral waffle that contains (and lies within ε of) Q . A waffle has *floor* polygons (forming a topological disk), *wall quadrangles* glued along floor edges, and *wall triangles* glued at floor vertices. Insetting the edges of Q on the tuck side bisects every dihedral angle, splitting each reflex (negative-curvature) angle into two convex ones, which makes every *waffle pocket* intrinsically convex.

Algorithm 2: COMPUTETUCKPROXY(Q, ε) — build the tuck proxy waffle (§3)

Input: Polyhedral manifold Q ; tolerance $\varepsilon > 0$.

Output: Tuck proxy T with walls of height $\leq \varepsilon$; parameter $0 < \varepsilon' < \varepsilon$ with no global self-intersection up to ε' .

- 1 Floor polygons of $T \leftarrow$ the facets of Q
 - 2 Choose ε' small enough that no collision event occurs while inseting
 - 3 **for each** edge e of Q on the tuck side **do**
 - 4 Inset e inward by ε' (start of the 3D straight skeleton)
 ▷ bisects the dihedral angle at e ; a reflex angle splits into two convex angles
 - 5 **for each** vertex v of Q **do**
 - 6 On a small sphere around v , connect the incident edge offsets into a cycle
 ▷ pocket bounded by floor polygon, two offset edges, one cycle edge;
 perimeter $\leq 360^\circ$
 - 7 Overlay a regular tetrahedron on the cycle and triangulate if needed
 ▷ makes the pocket intrinsically convex: edges $\leq 109.5^\circ$, perimeter
 $\leq 328.5^\circ < 360^\circ$
 - 8 Form wall quadrangles along floor edges and wall triangles at floor vertices (height $\leq \varepsilon$)
 - 9 **return** (T, ε')
-

§4 Waffle Pocket Squashing

Locally “squashes” each waffle pocket flat into the plane by adding material between polygons, only *increasing* bottom angles and preserving convexity, connectivity, and the cyclic order of wall polygons. The flattened top edges form a convex *cell*; its outward edge normals are the *cell perpendiculars* that will seed streams.

Algorithm 3: SQUASHWAFFLEPOCKETS(T, ε') — flatten every waffle pocket (§4)

Input: Tuck proxy T ; parameter ε' .

Output: For each pocket: a planar squashing (cell + squashed wall polygons) and its cell perpendiculars.

```
1 for each waffle pocket  $\pi$  (a floor vertex or a floor polygon) do
2   Vertex-unfold the wall polygons of  $\pi$  into the plane, spreading the angular gaps evenly
   between squashed wall polygons
3   for each gap do
4     Place a ray from the vertex along the angular bisector of the two adjacent squashed
     wall polygons (rounded to a polygon boundary if needed)
     ▷ consecutive rays sandwich each wall polygon at an angle  $< 180^\circ$ 
5   if  $\pi$  is a floor vertex then
6     Cell  $\leftarrow$  a small circle surrounding the vertex
7   else //  $\pi$  is a floor polygon
8     Cell  $\leftarrow$  offset of the floor polygon, corners smoothed with quadratic Bézier arcs
     ▷ wall quadrangles then squash into trapezoids
9   Clip the rays against the cell to bound the squashed wall polygons (strictly convex;
   bottom angles only increase)
10  for each cell edge do
11    Emit a cell perpendicular: outward normal starting at the midpoint of the bottom
    of the corresponding squashed wall polygon, oriented CCW
12 return the collection of squashings and cell perpendiculars
```

§5 Placing Streams

Embeds the *waffle dual* in the plane (Tutte embedding) inside a convex polygon P' , then connects the placed floor vertices/polygons by *streams*: thick *rivers* for wall quadrangles and zero-width *brooks* for wall triangles. Three nested disks around each node re-aim the stream directions to match the embedded dual without collisions; filleting makes streams C^1 with positive clearance δ .

Algorithm 4: PLACESTREAMS(T, ε', Σ) — route rivers and brooks (§5)

Input: Tuck proxy T ; parameter ε' ; squashing Σ .

Output: Convex polygon P' ; stream placement $\mathcal{S}$; clearance $\delta > 0$.

1 Part 1 — embed the waffle dual.

- 2 Compute a Tutte embedding of the waffle dual with its outside face, so boundary nodes lie on $\partial P'$ (P' convex)
- 3 Ensure every river-attached boundary node has room to be thickened parallel to its edge of P' without leaving it or hitting another boundary node

4 Part 2 — place nodes and connect.

5 for each waffle dual node x do

- 6 **if** x is a floor vertex **then** place a point
- 7 **else** place an isometric copy of the corresponding facet of Q

8 for each waffle dual edge do

- 9 **if** it is a wall quadrangle **then**
- 10 Draw a *river*: connect two equal-length floor-polygon edges and thicken by that edge length
- 11 **else**
- 12 Draw a *brook*: connect two points with a zero-width C^1 curve

13 for each node x of degree k do

- 14 Build nested disks $R_x \subset S_x \subset T_x$ (radii $r_x < s_x < t_x$):
- 15 R_x separates streams from the floor vertex/polygon
- 16 S_x bends streams to meet ∂S_x orthogonally
- 17 T_x twists streams onto incident dual-edge directions using k collision-free tracks
- 18 Rotate the floor polygon so one edge normal needs no twist; cut the $T_x \setminus S_x$ annulus there
- 19 Route each connection inside→outside in CCW order: outermost free track for leftward (CCW) links, innermost free track for rightward (CW) links

20 Fillet every sharp corner with a circular arc $\Rightarrow$ streams are C^1 with positive clearance δ

21 Scale up the Tutte embedding by a finite factor so all disks stay local

22 **return** ($P', \mathcal{S}, \delta$)

§6 Placing Sites

Places point, segment, and polygon *sites* on the paper so that their generalized Voronoi diagram — taken in the *geodesic vertex metric* — contracts to the waffle graph of T . Protective regions called *squashes* (pumpkins, jack-o'-lanterns, calabashes, cucumbers) keep new sites from corrupting earlier structure. The piece of paper P is then the convex hull of the resulting paired subcells.

Algorithm 5: PLACESITES($T, \varepsilon', \Sigma, \mathcal{S}$) — site placement (§6)

Input: Tuck proxy T ; parameter ε' ; squashing Σ ; streams $\mathcal{S}$.

Output: Piece of paper P and a set $\mathcal{V}$ of point/segment/polygon sites.

```
1 (1) Node sites.
2 for each brook node do
3   place a pumpkin (point site cluster)
4 for each placed floor polygon do
5   place a jack-o'-lantern
6   core = cell of the local squashing of that pocket
6 (2) Stream sites.
7 for each brook do
8   Lay point sites along it; insert a calabash between consecutive sites with flesh-polygon
9   angle = bottom angle of the brook's wall triangle
9 for each river do
10  Lay segment sites along it; insert a cucumber between consecutive sites with
11  flesh-polygon angles = bottom angles of the river's wall quadrangle
11 if a gap between consecutive sites lets another stream hit the squash then
12   Bisect the gap and add a site; subdivide until every squash point is  $\leq \frac{1}{2}\varepsilon'$  from the core
13 (3) Bank sites.
14 while some paper point is  $> \varepsilon'$  (Euclidean) from its nearest site do
15   Add a point site there
16   squashes  $\subseteq$  Minkowski sum of each site with an  $\varepsilon'$ -disk  $\Rightarrow$  terminates
17   off the squashes
16 (4) Define the paper.
17  $P \leftarrow$  convex hull (within  $P'$ ) of the paired subcells
18    $\triangleright$  equivalently: convex hull of the sites and the Voronoi diagram, clipped
19   to  $P'$ 
18 return  $(P, \mathcal{V})$ 
```

Invariants guaranteed by construction. (i) the Voronoi diagram contracts onto the modified dual — the waffle graph of T ; (ii) each Voronoi edge is straight and mirror-reflects its defining site vertices, forming *paired subcells* (quadrangular iff the edge realizes a wall quadrangle); (iii) every realizing Voronoi edge's subcell angles are $\geq$ the bottom angles of the corresponding wall polygon; (iv) every point of P lies within ε' of a site vertex.

§7 Voronoi Folding

Folds P into the tuck proxy T by a sequence of *folding steps*, each producing an *abstract waffle* (a metric polyhedral complex) and never separating layers already brought together. The final abstract waffle embeds isometrically onto T , giving the desired folding f .

Algorithm 6: $\text{VORONOI FOLDING}(T, P, \mathcal{V})$ — fold P onto the tuck proxy (§7)

Input: Tuck proxy T ; paper P ; sites/Voronoi $\mathcal{V}$.

Output: Watertight, seamless, ε -extra folding $f: P \rightarrow Q$ with $f(P) \subseteq T$.

```

1 (1) Initial fold into an abstract waffle  $A$ .
2  $G \leftarrow \{\text{Voronoi edge of each paired subcell}\} \cup \{\text{boundary edge of each unpaired subcell}\}$ 
3 for each Voronoi edge of  $G$  do
4   | Fold its two (mirror-image) paired subcells onto each other to doubly cover the
   |   matching wall polygon of  $A$ 
5 for each boundary edge of  $G$  do
6   | Fold the unpaired subcell to singly cover its wall polygon
   |    $\triangleright$  floor of  $A = \text{placed floor polygons} \Rightarrow \text{isometric to } Q$ ; each Voronoi cell
   |    $\rightarrow$  a pocket of  $A$ 
7 (2) Contract & merge non-realizing edges.
8 for each Voronoi edge not realizing a waffle-graph edge of  $T$  do
9   | Contract it: fold the wall triangle in half along its bisector and glue it onto an adjacent
   |   wall polygon of the same pocket
10  if several edges join the same two nodes then
11    | Crimp-fold the larger angles down to the smallest, stack the now isometric wall
    |   polygons, and collapse to a pocket
    |    $\triangleright$  leaves a simplified waffle graph whose dual is just the realizing paths
12 (3) Merge realizing edges.
13 for each dual path (sequence of wall polygons for one waffle-graph edge) do
14   | Crimp-fold their bottom angles to match the bottom angles of the corresponding wall
   |   polygon of  $T$ 
15   | Stack them as a single wall polygon and embed it isometrically into that wall polygon
   |   of  $T$ 
16  $f \leftarrow$  the resulting folded state  $\subseteq T$ 
17 return  $f$ 

```

Correctness of the output. **Seamless:** the floor polygons are isometric to Q and no other paper folds strictly interior to a facet. **Watertight:** as a point p moves CCW along ∂P and q moves CCW along ∂Q , the correspondence $f(p) = q$ on boundary sites (and q pinned to the nearest boundary vertex between sites) stays within Fréchet distance ε . **ε -extra:** every added wall feature has height $\leq \varepsilon$ and $f(P) \subseteq T$, so the Hausdorff distance from Q is $\leq \varepsilon$.